

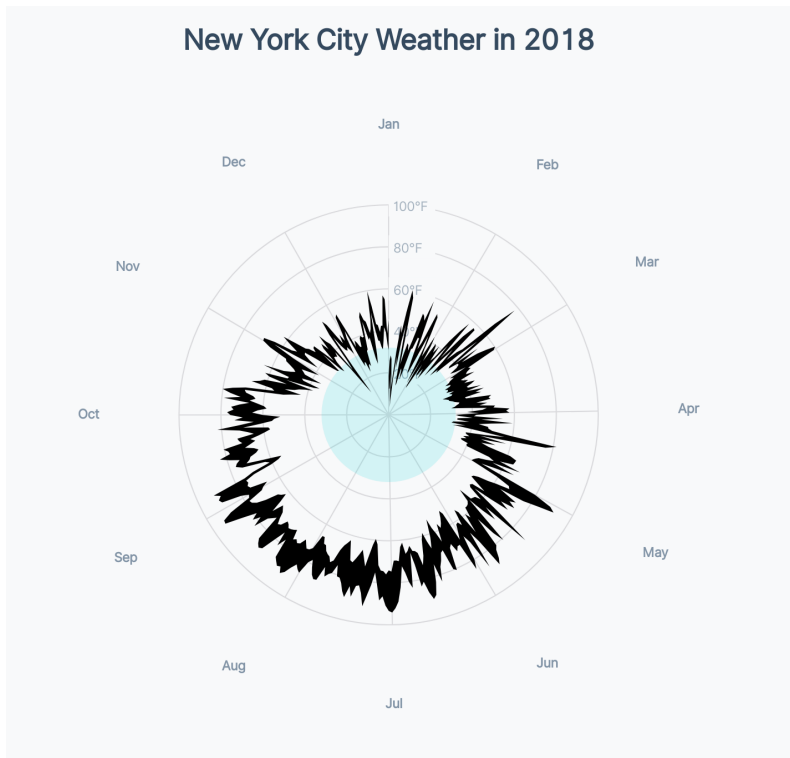


Chart with temperature area

Sometimes displaying a metric in multiple ways can help focus the viewer on it and also give them two ways to encode it. Let's also visualize the temperature with a gradient.

Let's create a gradient at the end of our **Draw canvas** step. Creating `<defs>` elements near the top helps organize our code – we'll know where to find elements that are re-useable.

This will look similar to drawing our legend gradient in **Chapter 6**, but we'll use a `<radialGradient>` instead of a `<linearGradient>`.

code/11-radar-weather-chart/completed/chart.js

```
58   const defs = wrapper.append("defs")
59
60   const gradientId = "temperature-gradient"
61   const gradient = defs.append("radialGradient")
62     .attr("id", gradientId)
63   const numberOfStops = 10
64   const gradientColorScale = d3.interpolateYlOrRd
65   d3.range(numberOfStops).forEach(i => {
66     gradient.append("stop")
67       .attr("offset", `${i * 100 / (numberOfStops - 1)}%`)
68       .attr("stop-color", gradientColorScale(i
69         / (numberOfStops - 1)
70       ))
71   })
```

Now we can use our gradientId to set our area's fill.

code/11-radar-weather-chart/completed/chart.js

```
180   const area = bounds.append("path")
181     .attr("class", "area")
182     .attr("d", areaGenerator(dataset))
183     .style("fill", `url(#${gradientId})`)
```

Great! Now we can see that our gradient re-enforces the relationship between distance from the origin and higher temperatures.

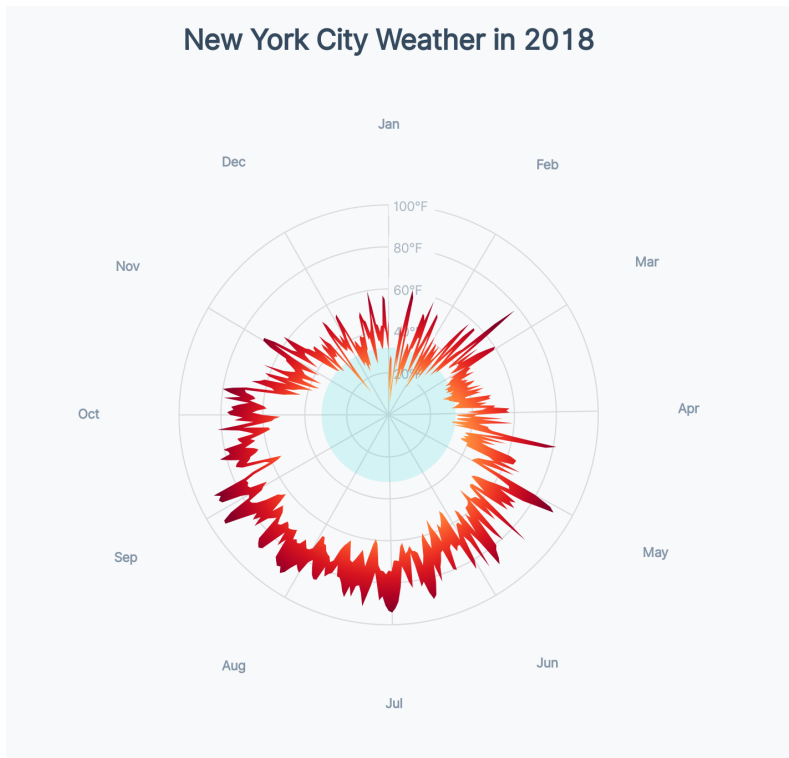


Chart with temperature area with gradient

Adding the UV index marks

Next, let's mark days that have a high UV index. But what does a "high UV index" mean? We'll need to make that decision ourselves – let's define a "high UV day" as any day with a UV index over 8.

code/11-radar-weather-chart/completed/chart.js

```
185  const uvIndexThreshold = 8
```

These kinds of decisions will come from your expertise as a subject matter expert. When setting a threshold like this in your own charts, think about what might be meaningful to the viewer.

Let's keep our code organized and keep our UV index lines within one group.

code/11-radar-weather-chart/completed/chart.js

186 **const** uvGroup = bounds.append("g")

We want to draw our UV lines just inside the edges of our radius – let's set their offset to 0.95.

code/11-radar-weather-chart/completed/chart.js

187 **const** uvOffset = 0.95

Next, let's draw one `<line>` per day over our threshold, drawing the outside edge just outside of our chart's radius.

code/11-radar-weather-chart/completed/chart.js

```
188     const highUvDays = uvGroup.selectAll("line")
189         .data(dataset.filter(d => uvAccessor(d) > uvIndexThreshold))
190         .enter().append("line")
191             .attr("class", "uv-line")
192             .attr("x1", d => getXFromDataPoint(d, uvOffset))
193             .attr("x2", d => getXFromDataPoint(d, uvOffset + 0.1))
194             .attr("y1", d => getYFromDataPoint(d, uvOffset))
195             .attr("y2", d => getYFromDataPoint(d, uvOffset + 0.1))
```

We won't be able to see our `<line>`s until we give them a stroke – let's add a stroke color and width in our `styles.css` file.

```
.uv-line {
  stroke: #feca57;
  stroke-width: 2;
}
```

Now we can see that all of the days with high UV index are between April and September, with the highest density around July.

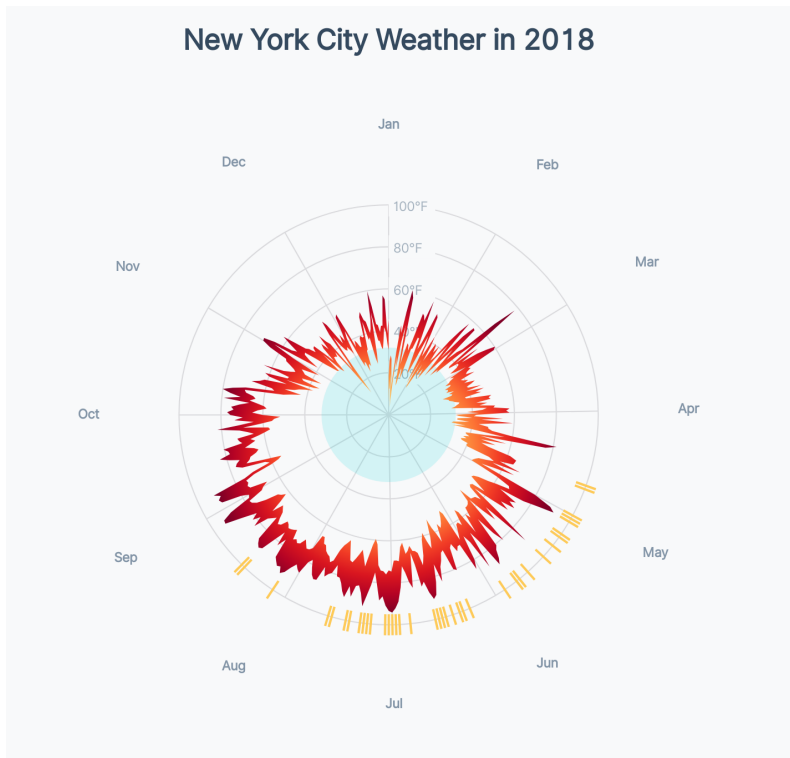
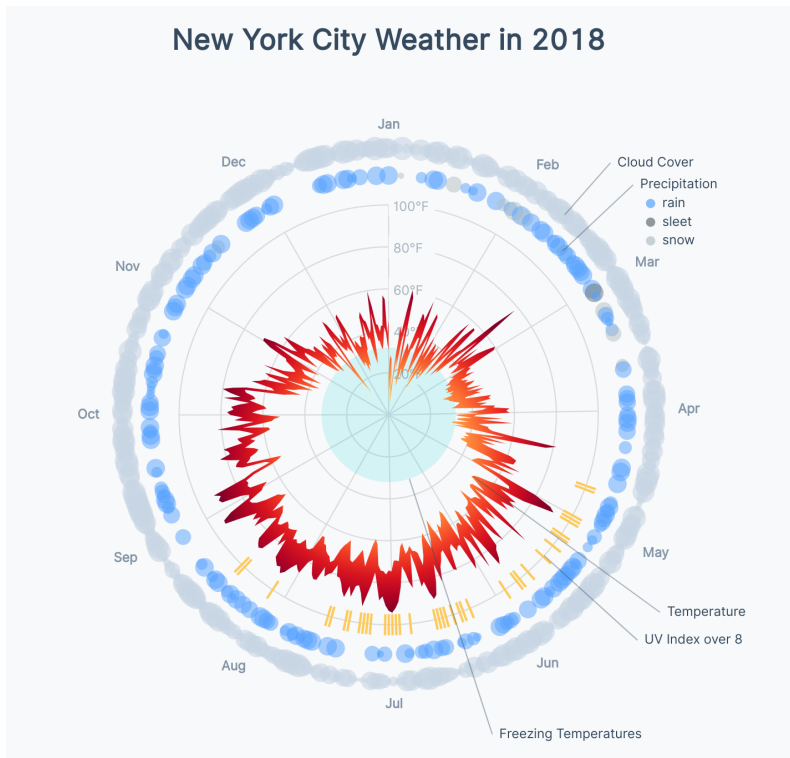


Chart with uv index lines

Adding the cloud cover bubbles

Next, let's add the gray circles around our chart that show how much cloud cover each day has. As a reminder, this is what our final chart will look like:



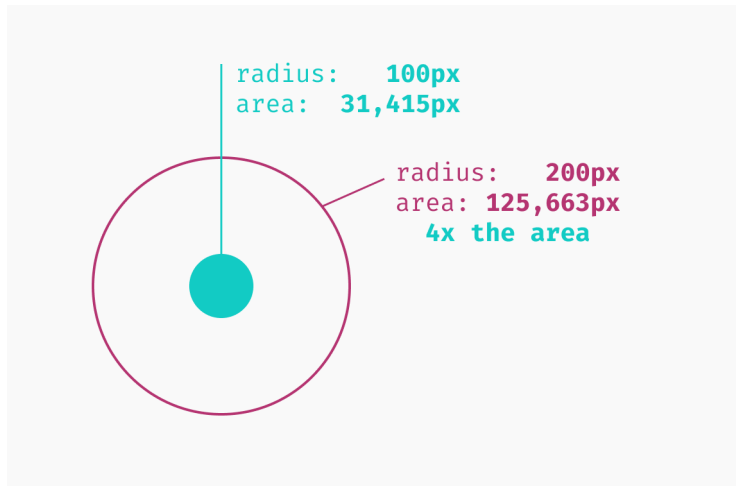
Radar Weather Chart

The *radius* of each of our circles will depend on the amount of cloud cover.

As an example of how we can use different **dimensions** to visualize a metric (like we learned in **Chapter 7**), we could have encoded the amount of cloud cover as the color of each circle, instead of the size. In this case, size works better because we're already using a color scale for our temperature *and* our precipitation type. To prevent from distracting the eye with too many colors, we'll vary our cloud cover circles by *size* instead.

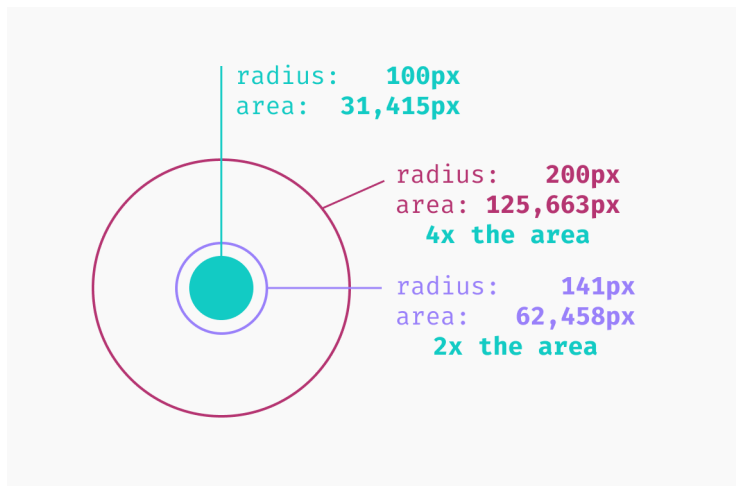
One caveat with visualizing a *linear scale* with a circle's size is that circles' *areas* and *radii* scale at different rates. Let's take a circle with a radius of 100px as an example. If we multiply its radius by 2, we'll get a circle with a radius of 200. However, the

circle grows in every direction, making this larger circle cover *four times* as much space.



Circle example

Instead, we'll want a circle with a radius of 141 pixels to create a circle that is twice as *large* as our original circle.



Circle example, part 2

Since we, as humans, judge a circle by the *amount of space it takes up*, instead of

how wide it is, we need a way to size our circles by their area instead of their radii. But `<circle>` elements are sized with their `r` attribute, so we need a way to scale our radii so that our areas scale linearly.

The *area* of a circle is the *radius* multiplied by π , then squared. If we flip this equation around, we'll find that the *radius* of a circle is the square root of a circle's *area*, divided by π .

$$\text{area} = \pi r^2$$
$$\text{radius} = \frac{\sqrt{\text{area}}}{\pi}$$

area and radius formulas

Since π is a constant, we can represent the relationship simply by using a square root scale. How convenient!

Whenever we're scaling a circle's radius, we'll want to use `d3.scaleSqrt()`⁷² instead of `d3.scaleLinear()`⁷³ to keep the circles' areas scaling proportionally.

Let's create our cloud cover radius scale, making our circles' radii range from 1 to 10 pixels.

We'll write this code at the end of our **Create scales** step.

⁷²<https://github.com/d3/d3-scale#scaleSqrt>

⁷³<https://github.com/d3/d3-scale#scaleLinear>

code/11-radar-weather-chart/completed/chart.js

```
97   const cloudRadiusScale = d3.scaleSqrt()  
98     .domain(d3.extent(dataset, cloudAccessor))  
99     .range([1, 10])
```

At the end of our **Draw data** step, let's create a new `<g>` to contain our cloud circles, then declare their offset from the center of our chart.

code/11-radar-weather-chart/completed/chart.js

```
197   const cloudGroup = bounds.append("g")  
198   const cloudOffset = 1.27
```

Now we can draw one circle per day, setting each circle's radius with our new `cloudRadiusScale`.

code/11-radar-weather-chart/completed/chart.js

```
199   const cloudDots = cloudGroup.selectAll("circle")  
200     .data(dataset)  
201     .enter().append("circle")  
202     .attr("class", "cloud-dot")  
203     .attr("cx", d => getXFromDataPoint(d, cloudOffset))  
204     .attr("cy", d => getYFromDataPoint(d, cloudOffset))  
205     .attr("r", d => cloudRadiusScale(cloudAccessor(d)))
```

Great! Now we can see a ring of “clouds” around the outside of our chart.

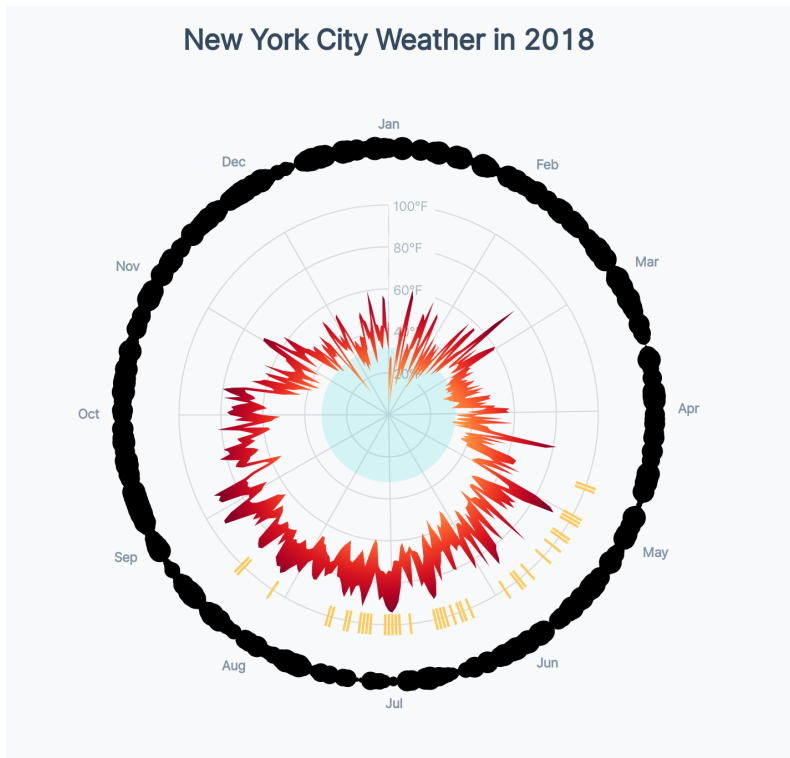


Chart with cloud bubbles

Let's set their `fill` color in our `styles.css` file, dimming them so they are a more natural "cloud" color, and so they don't visually dominate our chart.

```
.cloud-dot {  
  fill: #c8d6e5;  
}
```

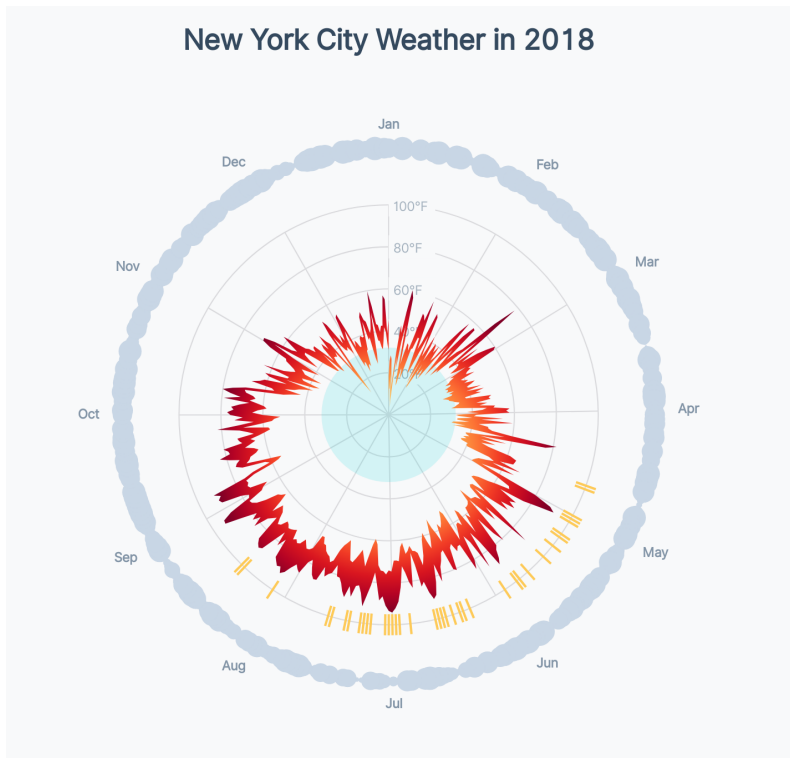


Chart with cloud bubbles, colored

Let's also make our cloud circles somewhat translucent, so that larger circles don't completely cover their smaller neighbors.

```
.cloud-dot {  
  fill: #c8d6e5;  
  opacity: 0.6;  
}
```

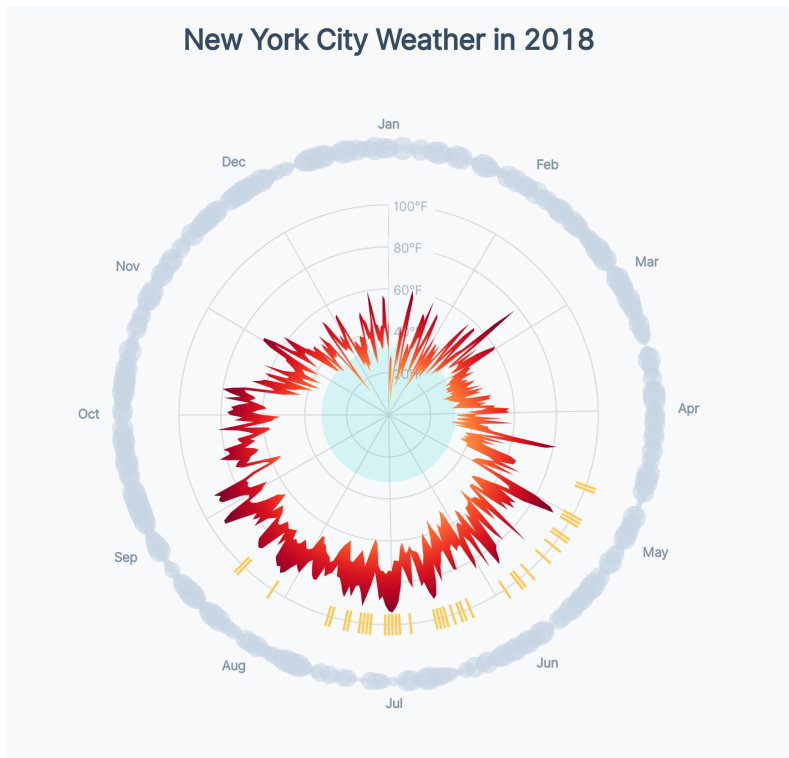


Chart with cloud bubbles, translucent

Adding the precipitation bubbles

Next, we want to add a row of bubbles corresponding to each day's precipitation. We have two relevant metrics: *probability* of precipitation and *type* of precipitation. Let's visualize the *probability* with the size of the bubble and the *type* with the color of the bubble. This way, we can play to both dimensions' strengths – a large amount of *blue* will correspond to a *high probability* of rain. And we won't see a lot of a color if we're not confident that it did precipitate.

To start, we'll create a scale to convert the probability of precipitation to the radius of a bubble. We'll make these circles a little smaller than our cloud circles, since they're closer to the middle of our circle (and thus have less space).

This code will go at the end of our **Create scales** step.

code/11-radar-weather-chart/completed/chart.js

```
101  const precipitationRadiusScale = d3.scaleSqrt()  
102    .domain(d3.extent(dataset, precipitationProbabilityAccessor))  
103    .range([1, 8])
```

Next, we'll list out the types of precipitation in our dataset, then create a color scale mapping those types to different colors. We'll want to use an **ordinal** scale, since this is an **ordinal** metric, and can be placed in categories with a natural order (remember the types of data we learned in **Chapter 7**?).

code/11-radar-weather-chart/completed/chart.js

```
104  const precipitationTypes = ["rain", "sleet", "snow"]  
105  const precipitationTypeColorScale = d3.scaleOrdinal()  
106    .domain(precipitationTypes)  
107    .range(["#54a0ff", "#636e72", "#b2bec3"])
```

Scrolling back down to the end of our **Draw data** step, we'll draw our circles similarly to how we drew our cloud circles. This time, we'll use our `precipitationTypeColorScale` to set each circle's fill color.

`code/11-radar-weather-chart/completed/chart.js`

```
207  const precipitationGroup = bounds.append("g")
208  const precipitationOffset = 1.14
209  const precipitationDots = precipitationGroup.selectAll("circle")
210    .data(dataset.filter(precipitationTypeAccessor))
211    .enter().append("circle")
212      .attr("class", "precipitation-dot")
213      .attr("cx", d => getXFromDataPoint(d, precipitationOffset))
214      .attr("cy", d => getYFromDataPoint(d, precipitationOffset))
215      .attr("r", d => precipitationRadiusScale(
216        precipitationProbabilityAccessor(d)
217      ))
218      .style("fill", d => precipitationTypeColorScale(
219        precipitationTypeAccessor(d)
220      ))
```

Great! Now we can see our inner circle of precipitation bubbles.

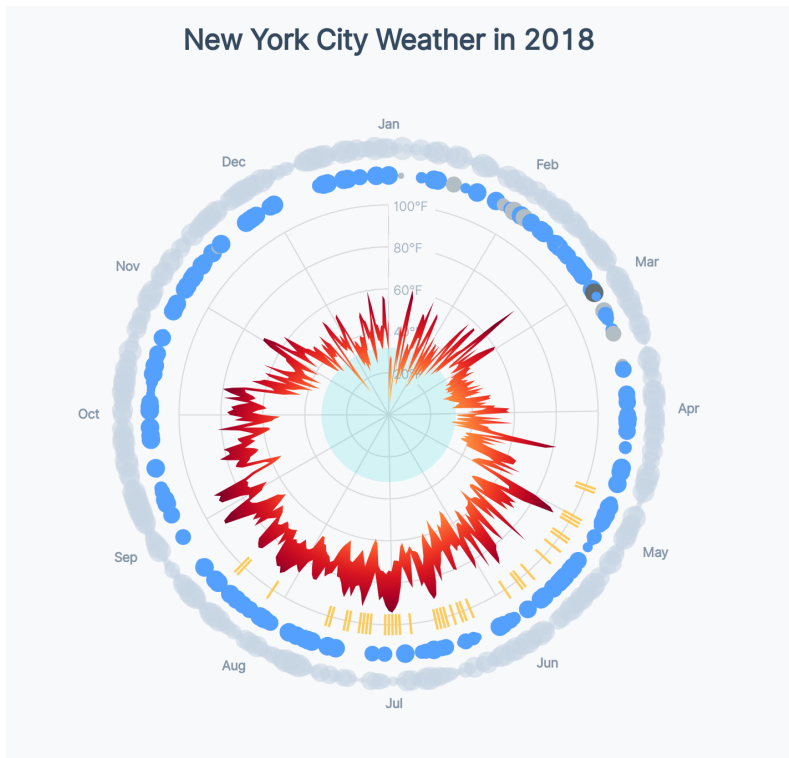


Chart with precipitation bubbles

Let's make these bubbles translucent as well, in our `styles.css` file:

```
.precipitation-dot {  
  opacity: 0.5;  
}
```

That's better!

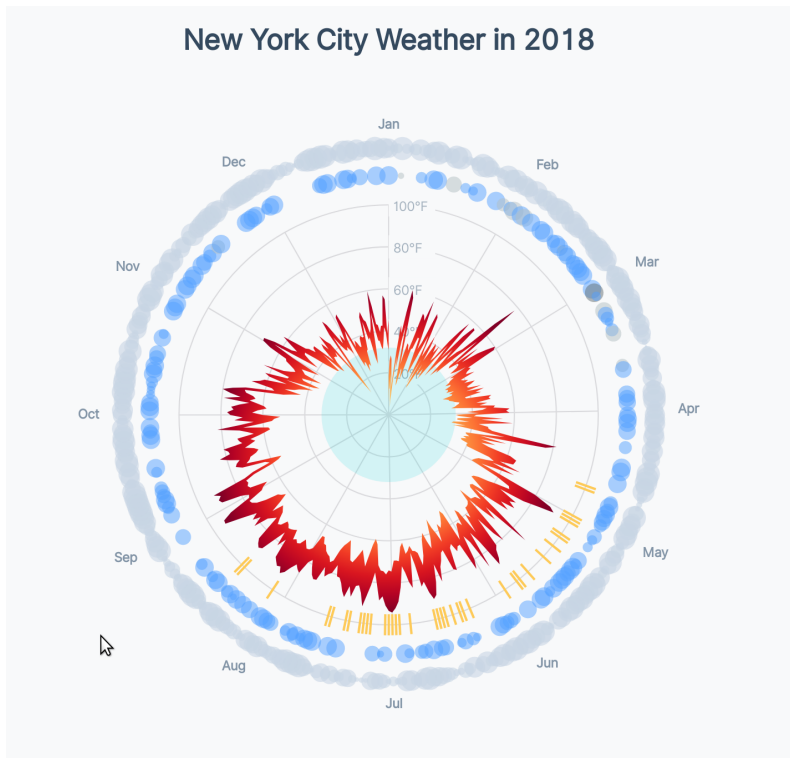


Chart with precipitation bubbles, translucent

Adding annotations

Let's take a step back and look at our charts through a new viewer's eyes.

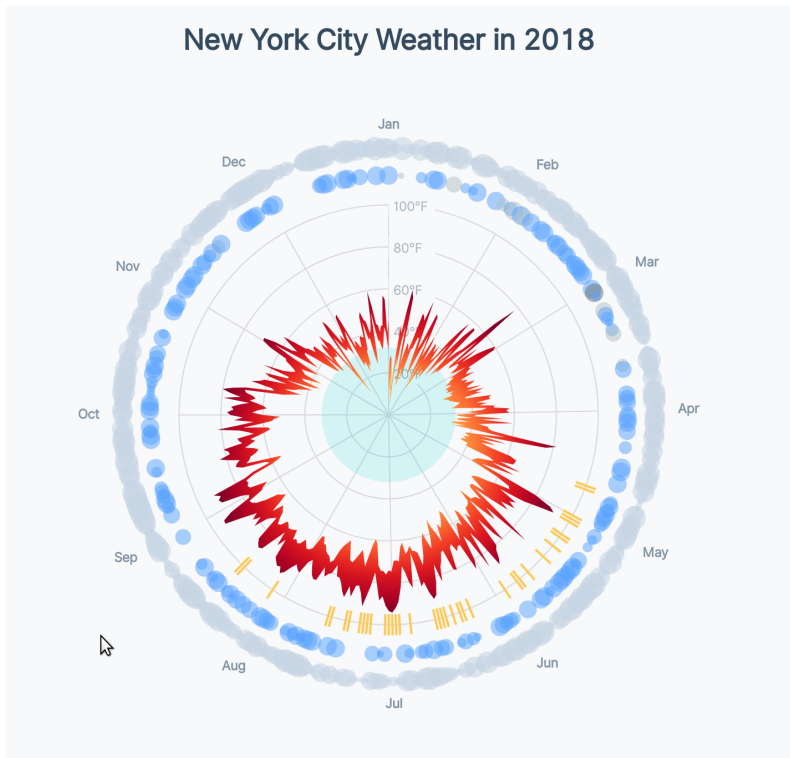


Chart with precipitation bubbles, translucent

There's a lot going on and not much explanation. A new viewer might wonder: what does this blue dot represent? What are these yellow slashes?

Let's add some annotations to help orient a new viewer. We have a lot of things that need explanation, so let's start by creating a function that will draw an annotation, give three parameters:

1. the **angle** around our circle
2. the **offset** from the center of our circle to start our line at
3. the **text** that we want to display

We'll also create a `<g>` element to contain all of our annotations.

```
const annotationGroup = bounds.append("g")
```

```
const drawAnnotation = (angle, offset, text) => {  
}
```

Within our `drawAnnotation()` function, we want to draw a `<line>` that extends from our `offset` to a set distance from the center of our chart. Let's draw our lines out to 1.6 times our circle's radius, just outside of our cloud bubbles. We'll also want a `<text>` element to display the text of our annotation, which we'll draw at the outer end of our `<line>`.

code/11-radar-weather-chart/completed/chart.js

```
227  const drawAnnotation = (angle, offset, text) => {  
228    const [x1, y1] = getCoordinatesForAngle(angle, offset)  
229    const [x2, y2] = getCoordinatesForAngle(angle, 1.6)  
230  
231    annotationGroup.append("line")  
232      .attr("class", "annotation-line")  
233      .attr("x1", x1)  
234      .attr("x2", x2)  
235      .attr("y1", y1)  
236      .attr("y2", y2)  
237  
238    annotationGroup.append("text")  
239      .attr("class", "annotation-text")  
240      .attr("x", x2 + 6)  
241      .attr("y", y2)  
242      .text(text)  
243  }
```

We'll want our `<line>` to have a light stroke and our `<text>` to be vertically centered with the end of our `<line>`. Let's add those styles to our `styles.css` file.

```
.annotation-line {  
  stroke: #34495e;  
  opacity: 0.4;  
}  
  
.annotation-text {  
  fill: #34495e;  
  font-size: 0.7em;  
  dominant-baseline: middle;  
}
```

Going back to our `chart.js` file, let's draw our first two annotations. To keep the top of our chart as clean as possible, let's create an annotation for the outer two data elements: cloud and precipitation bubbles.

We'll want to draw these annotations in the *top right* of our chart, to prevent from stealing the show too early. If our annotations were in the top left, viewers might read them first (since English text usually runs from left-to-right, top-to-bottom). We'll set the angle of these two annotations around $\pi / 4$, which is one-eighth of a turn around our chart.

And for our annotations' offset, we can use the offsets we defined when we drew each set of bubbles.

```
drawAnnotation(Math.PI * 0.23, cloudOffset, "Cloud Cover")  
drawAnnotation(Math.PI * 0.26, precipitationOffset, "Precipitation")
```

Wonderful! Our annotations fit in between two of our month labels, preventing any overlap.

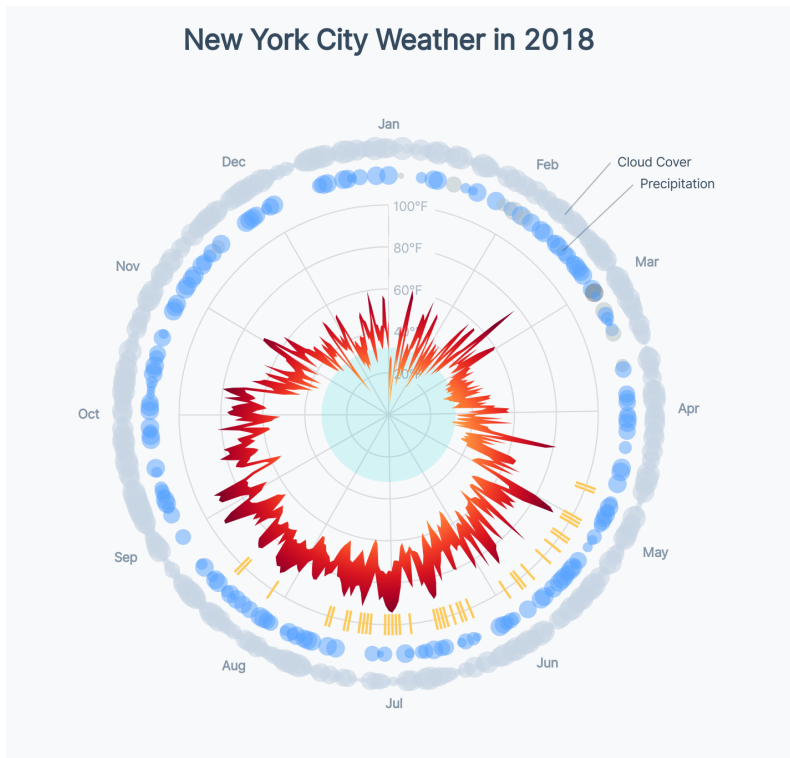


Chart with two annotations

We'll draw the rest of our annotations in the *bottom right* of our chart, making sure to tell our viewers what the exact UV index threshold is.

```
drawAnnotation(Math.PI * 0.734, uvOffset + 0.05, `UV Index over ${
  uvIndexThreshold
}`)
drawAnnotation(Math.PI * 0.7, 0.5, "Temperature")
if (containsFreezing) {
  drawAnnotation(
    Math.PI * 0.9,
    radiusScale(32) / dimensions.boundedRadius,
    "Freezing Temperatures"
  )
}
```


Note that we had to convert our freezing point into a value *relative to our bounded radius*, since our `drawAnnotation()` function takes an offset instead of a radius value.

Hmm, our longer annotation labels are cut off.

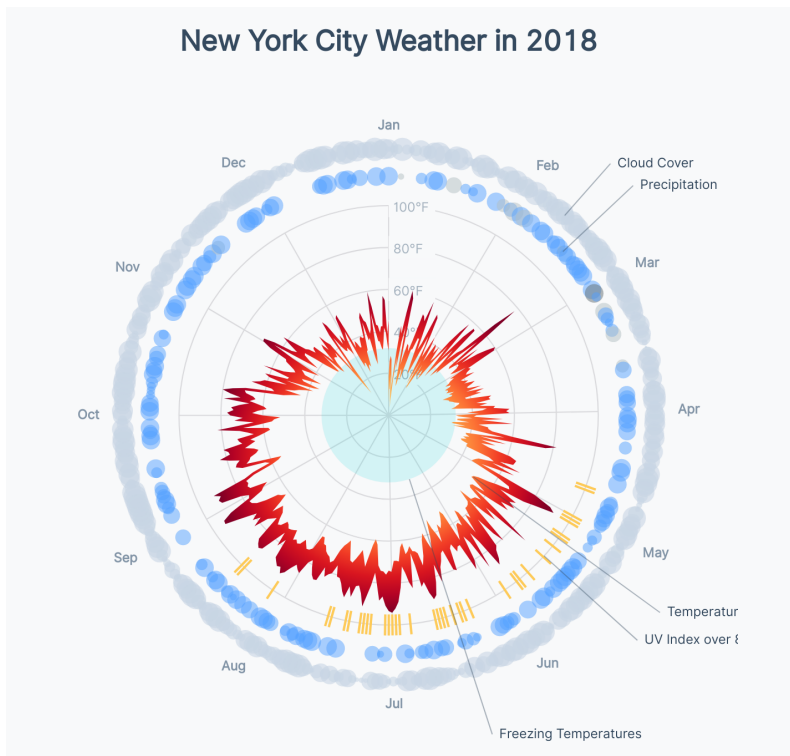


Chart with all annotations

We *could* increase the size of our right margin, but that would un-center our chart within our **wrapper**. Not a big deal, but let's look at an alternative: prevent overflowing svg elements from being clipped.

In our `styles.css` file, let's change the `overflow` property of our `svg` from the default of `hidden`.

```
svg {
  overflow: visible;
}
```

Easy peasy! Now we can see the end of our annotations. Be careful, though, when using this workaround in complicated pages – you don’t want your chart to run into other elements on your page!

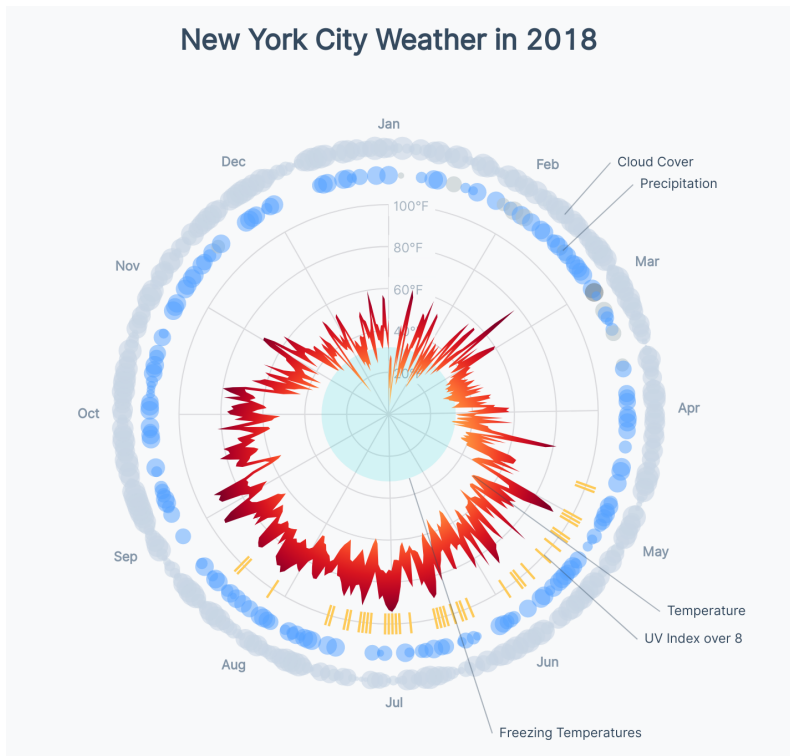


Chart with all annotations

This looks great, but feel free to play around with the angle of your annotations. Maybe you would group all of your annotation labels in the *top right*. Keep in mind that irregular shapes waste more space in many page layouts.

If we again view our chart with a new viewer’s eyes, each part is way more clear! We are missing one thing, though: the precipitation type colors are still un-labeled.

Let’s loop over each of our precipitation types, creating one `<circle>` to show the

color and one <text> element to label the color.

```
precipitationTypes.forEach((precipitationType, index) => {  
  const labelCoordinates = getCoordinatesForAngle(Math.PI * 0.26, 1.6)  
  annotationGroup.append("circle")  
    .attr("cx", labelCoordinates[0] + 15)  
    .attr("cy", labelCoordinates[1] + (16 * (index + 1)))  
    .attr("r", 4)  
    .style("opacity", 0.7)  
    .attr("fill", precipitationTypeColorScale(precipitationType))  
  annotationGroup.append("text")  
    .attr("class", "annotation-text")  
    .attr("x", labelCoordinates[0] + 25)  
    .attr("y", labelCoordinates[1] + (16 * (index + 1)))  
    .text(precipitationType)  
})
```

Great! Now a new viewer is quickly oriented and can figure out what each data element represents.

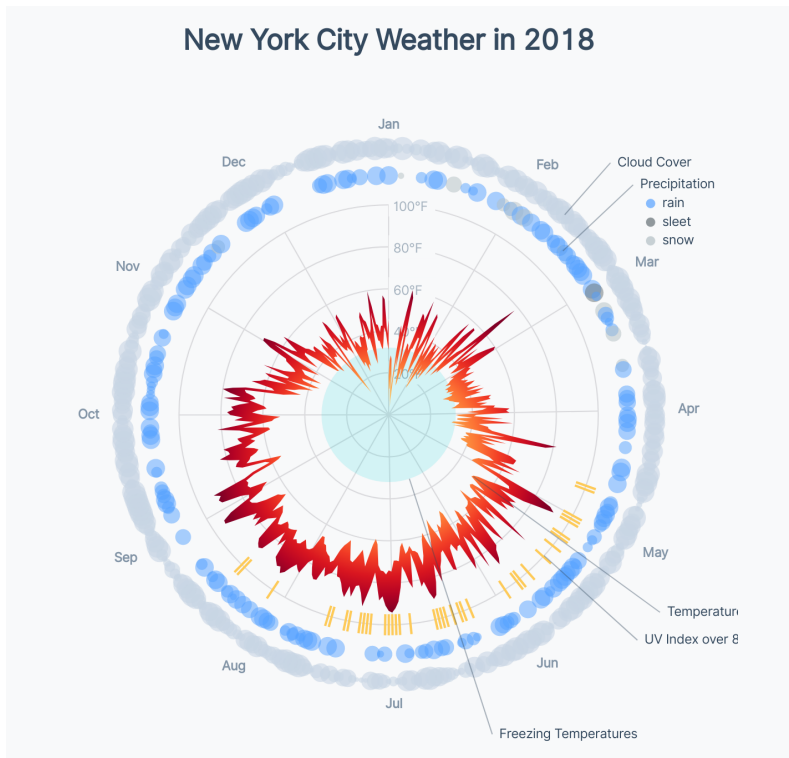


Chart with precipitation types annotation

Adding the tooltip

Now for the fun part: adding interactions! Although our viewers can orient themselves to the different parts of our chart, we also want them to be able to dig in and view details about a particular day.

Let's add a tooltip that shows up when the user hovered anywhere over the chart. We'll want to start by adding a listener element that covers our whole chart and initializing our mouse move events.

```
const listenerCircle = bounds.append("circle")  
  .attr("class", "listener-circle")  
  .attr("r", dimensions.width / 2)  
  .on("mousemove", onMouseMove)  
  .on("mouseleave", onMouseLeave)
```

```
function onMouseMove(e) {  
}
```

```
function onMouseLeave() {  
}
```

Perfect, the black area covers exactly where we want any movement to trigger a tooltip.

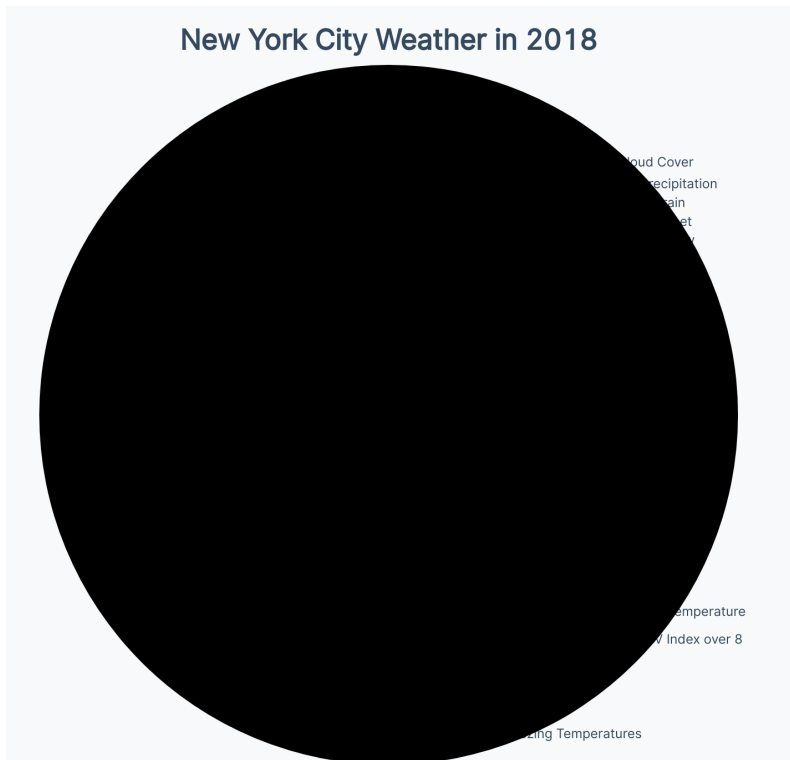


Chart with listener circle

Let's hide our listener by making its `fill` transparent.

```
.listener-circle {  
  fill: transparent;  
}
```

Next, we'll need to create our tooltip element in our `index.html` file, with a spot for each of our hovered over day's metrics to be displayed.

```
<div id="tooltip" class="tooltip">
  <div id="tooltip-date" class="tooltip-date"></div>
  <div id="tooltip-temperature" class="tooltip-temperature">
    <span id="tooltip-temperature-min"></span>
    -
    <span id="tooltip-temperature-max"></span>
  </div>
  <div class="tooltip-metric tooltip-uv">
    <div>UV Index</div>
    <div id="tooltip-uv"></div>
  </div>
  <div class="tooltip-metric tooltip-cloud">
    <div>Cloud Cover</div>
    <div id="tooltip-cloud"></div>
  </div>
  <div class="tooltip-metric tooltip-precipitation">
    <div>Precipitation Probability</div>
    <div id="tooltip-precipitation"></div>
  </div>
  <div class="tooltip-metric tooltip-precipitation-type">
    <div>Precipitation Type</div>
    <div id="tooltip-precipitation-type"></div>
  </div>
</div>
```

Let's also add our tooltip styles in our `styles.css` file, remembering to hide our tooltip and to give our **wrapper** a position to create a new context.

```
.wrapper {  
  position: relative;  
}  
  
.tooltip {  
  opacity: 0;  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 15em;  
  padding: 0.6em 1em;  
  background: #fff;  
  text-align: center;  
  line-height: 1.4em;  
  font-size: 0.9em;  
  border: 1px solid #ddd;  
  z-index: 10;  
  pointer-events: none;  
}  
  
.tooltip-date {  
  margin-bottom: 0.2em;  
  font-weight: 600;  
  font-size: 1.1em;  
  line-height: 1.4em;  
}  
  
.tooltip-temperature {  
  font-feature-settings: 'tnum' 1;  
}  
  
.tooltip-metric {  
  display: flex;  
  justify-content: space-between;  
  width: 100%;  
  font-size: 0.8em;
```



```

    line-height: 1.3em;
    transition: all 0.1s ease-out;
}

.tooltip-metric div:first-child {
    font-weight: 800;
    padding-right: 1em;
}

.tooltip-metric div:nth-child(2) {
    font-feature-settings: 'tnum' 1;
}

.tooltip-cloud {
    color: #8395a7;
}

.tooltip-uv {
    color: #feca57;
}

```

Switching back in our `chart.js` file, we'll want to grab our tooltip element to reference later, and also make a `<path>` element to highlight the hovered over day.

```

const tooltip = d3.select("#tooltip")
const tooltipLine = bounds.append("path")
    .attr("class", "tooltip-line")

```

Now we can fill out our `onMouseMove()` function. Let's start by grabbing the `x` and `y` position of our cursor, using `d3.mouse()`.

```

function onMouseMove(e) {
    const [x, y] = d3.mouse(this)
    // ...

```

We have our mouse position, but we need to know the *angle* from the chart origin.

How do we convert from an $[x, y]$ position to an angle? We'll need to use an inverse trigonometric function: `atan2`. If you're curious, [read more about atan2 here](#)⁷⁴.

code/11-radar-weather-chart/completed/chart.js

```
293   const getAngleFromCoordinates = (x, y) => Math.atan2(y, x)
```

Remember that these trigometric functions originate around the *horizontal, right* plane of our circle. Let's rotate the resulting angle back one-quarter turn around the circle to match our date scale.

```
let angle = getAngleFromCoordinates(x, y) + Math.PI / 2
```

To keep our angles *positive*, we'll want to rotate any *negative* angles around our circle by one full turn, so they fit on our angleScale.

```
if (angle < 0) angle = (Math.PI * 2) + angle
```

We want to draw a line to highlight the date we're hovering, but it needs to *increase in width* as it gets further from the center of our circle. To create this shape, we'll use `d3.arc()`⁷⁵, which is the `arc` version of the line generators we've been using (`d3.line()`). We can use the `.innerRadius()` and `.outerRadius()` methods to tell it how *long* we want our arc to be, and the `.startAngle()` and `.endAngle()` methods to tell it how *wide* we want our arc to be.

code/11-radar-weather-chart/completed/chart.js

```
297   const tooltipArcGenerator = d3.arc()
298     .innerRadius(0)
299     .outerRadius(dimensions.boundedRadius * 1.6)
300     .startAngle(angle - 0.015)
301     .endAngle(angle + 0.015)
```

Now we can use our new arc generator to create the `d` attribute for our tooltip line.

⁷⁴<https://en.wikipedia.org/wiki/Atan2>

⁷⁵<https://github.com/d3/d3-shape#arcs>

```
tooltipLine.attr("d", tooltipArcGenerator())  
    .style("opacity", 1)
```

Perfect! Now we have a line that follows our cursor around the center of our circle.

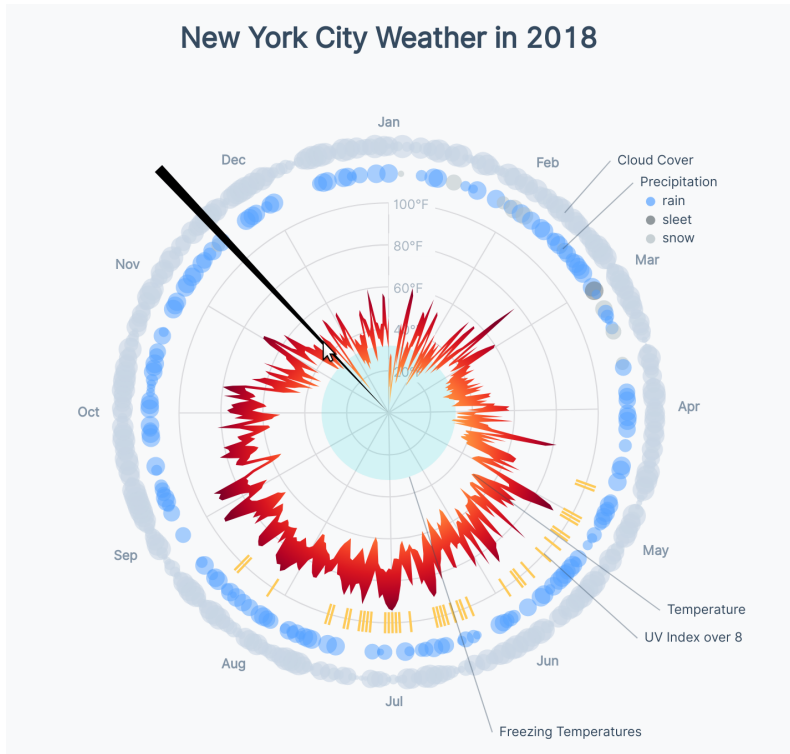


Chart with tooltip line

Let's lighten the line in our `styles.css` file to prevent it from covering the data we want to highlight. We can use `mix-blend-mode: multiple` to make the covered data elements stand out a little.

```

.tooltip-line {
  fill: #8395a7;
  fill-opacity: 0.2;
  mix-blend-mode: multiply;
  pointer-events: none;
}

```

Much better!

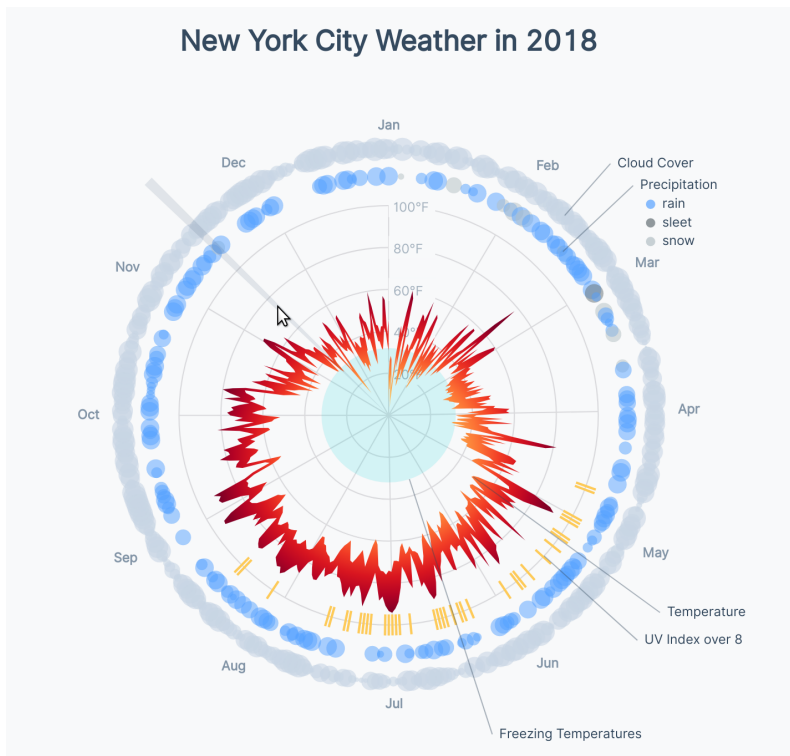


Chart with tooltip line, lightened

Next, we'll want to position our tooltip at the end of our line. First, we'll grab the `[x, y]` coordinates of this point.

code/11-radar-weather-chart/completed/chart.js

```
306      const outerCoordinates = getCoordinatesForAngle(angle, 1.6)
```

Using these coordinates, we'll set the transform CSS property of our tooltip. We have some fancy math here, using the CSS `calc()` function to choose which side of our tooltip to anchor to the outerCoordinate, based on where we are around the circle. We don't want our tooltip to cover our chart!

Try to work through each line to figure out what is going on, and inspect the tooltip in the **Elements** tab of your dev tools to see the resulting transform value.

```
tooltip.style("opacity", 1)
  .style("transform", `translate(calc(${
    outerCoordinates[0] < - 50 ? "40px - 100" :
    outerCoordinates[0] > 50 ? "-40px + 0" :
    "-50"
  })% + ${
    outerCoordinates[0]
    + dimensions.margin.top
    + dimensions.boundedRadius
  }px), calc(${
    outerCoordinates[1] < - 50 ? "40px - 100" :
    outerCoordinates[1] > 50 ? "-40px + 0" :
    "-50"
  })% + ${
    outerCoordinates[1]
    + dimensions.margin.top
    + dimensions.boundedRadius
  }px))`)
```

Wonderful! Now our tooltip follows the end of our tooltip line when we move our mouse around our chart.

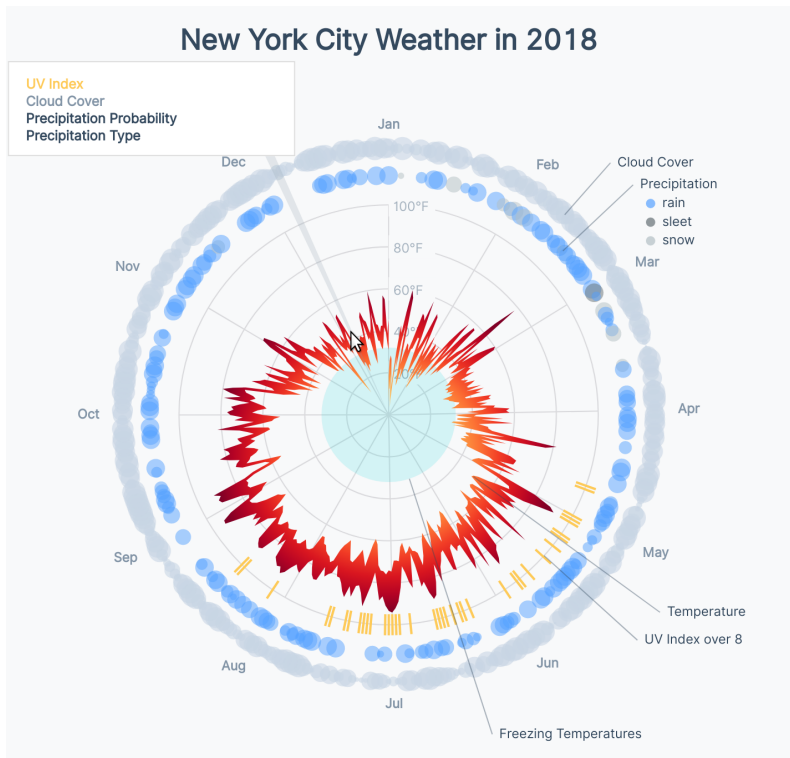


Chart with positioned tooltip

Next, we need to update the text of our tooltip to show information about the date we're hovering over. We can use the `.invert()` method of our `angleScale()` to convert backwards, from its **range** dimension (angle) to its **domain** dimension (date).

```
const date = angleScale.invert(angle)
```

If we format this date similarly to the dates in our dataset, we can look for a data point with the same date string.

```
const dateString = d3.timeFormat("%Y-%m-%d")(date)
const dataPoint = dataset.filter(d => d.date == dateString)[0]
```

If no such date exists, we should exit this function early. This should never happen, but is possible with a dataset that skips dates.

```
if (!dataPoint) return
```

Now that we have the data for the date we're hovering over, we can populate the text of our tooltip.

```
tooltip.select("#tooltip-temperature-min")
  .html(`${d3.format(".1f")(temperatureMinAccessor(dataPoint))}°F`)
tooltip.select("#tooltip-temperature-max")
  .html(`${d3.format(".1f")(temperatureMaxAccessor(dataPoint))}°F`)
tooltip.select("#tooltip-uv")
  .text(uvAccessor(dataPoint))
tooltip.select("#tooltip-cloud")
  .text(cloudAccessor(dataPoint))
tooltip.select("#tooltip-precipitation")
  .text(d3.format(".0%")(precipitationProbabilityAccessor(dataPoint)))
tooltip.select("#tooltip-precipitation-type")
  .text(precipitationTypeAccessor(dataPoint))
tooltip.select(".tooltip-precipitation-type")
  .style("color", precipitationTypeAccessor(dataPoint)
    ? precipitationTypeColorScale(precipitationTypeAccessor(dataPoint))
    : "#dadadd")
```

Notice that we're also setting the color of our precipitation type label and value, re-enforcing the relationship between the precipitation type and its color.

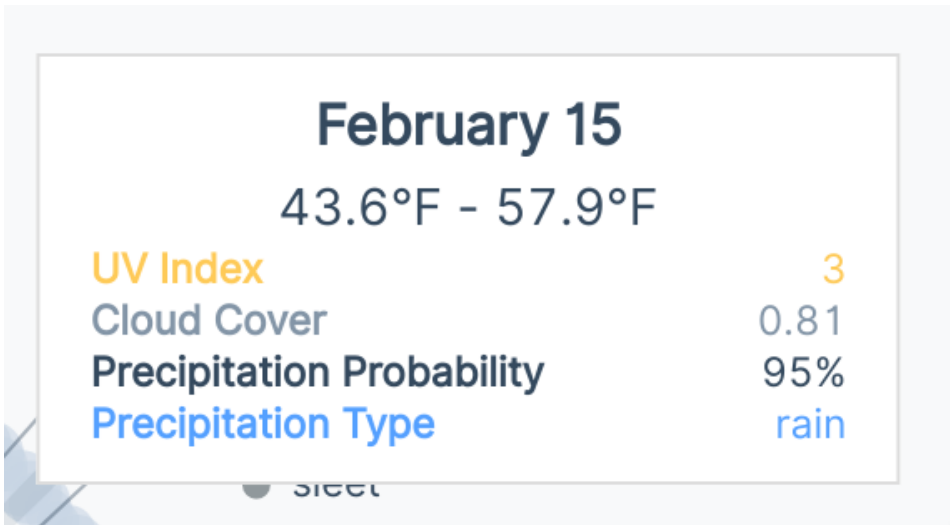


Chart with filled out tooltip

Let's take this one step further! We're using a gradient of colors to show what temperatures each day spans. At the end of our **Create scales** step, let's create a new scale that maps temperatures to the gradient scale we're using.

code/11-radar-weather-chart/completed/chart.js

```
109  const temperatureColorScale = d3.scaleSequential()  
110    .domain(d3.extent([  
111      ...dataset.map(temperatureMaxAccessor),  
112      ...dataset.map(temperatureMinAccessor),  
113    ]))  
114    .interpolator(gradientColorScale)
```

We're using a [sequential scale](https://d3js.org/d3-scale#sequential-scales)^a here instead of a linear scale because we want to use one of d3's built-in color scales (`d3.interpolateYlOrRd`) as an `.interpolator()` instead of specifying a range.

^a<https://github.com/d3/d3-scale#sequential-scales>

Now we can use this scale to color the minimum and maximum temperatures for our hovered date.

```
tooltip.select("#tooltip-temperature-min")
    .style("color", temperatureColorScale(
        temperatureMinAccessor(dataPoint)
    ))
tooltip.select("#tooltip-temperature-max")
    .style("color", temperatureColorScale(
        temperatureMaxAccessor(dataPoint)
    ))
```

And voila! Our tooltip is chock full of helpful information, and it also helps re-enforce some of the data visualization in our main chart.

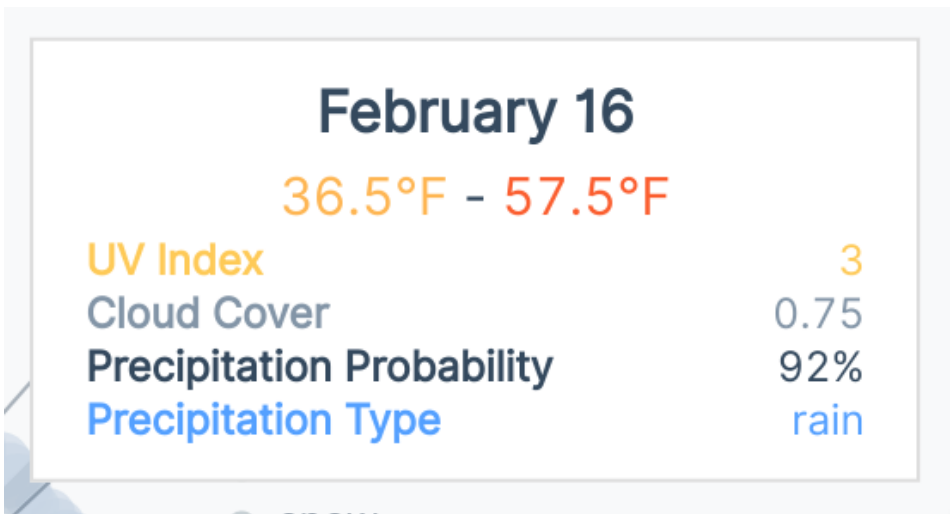
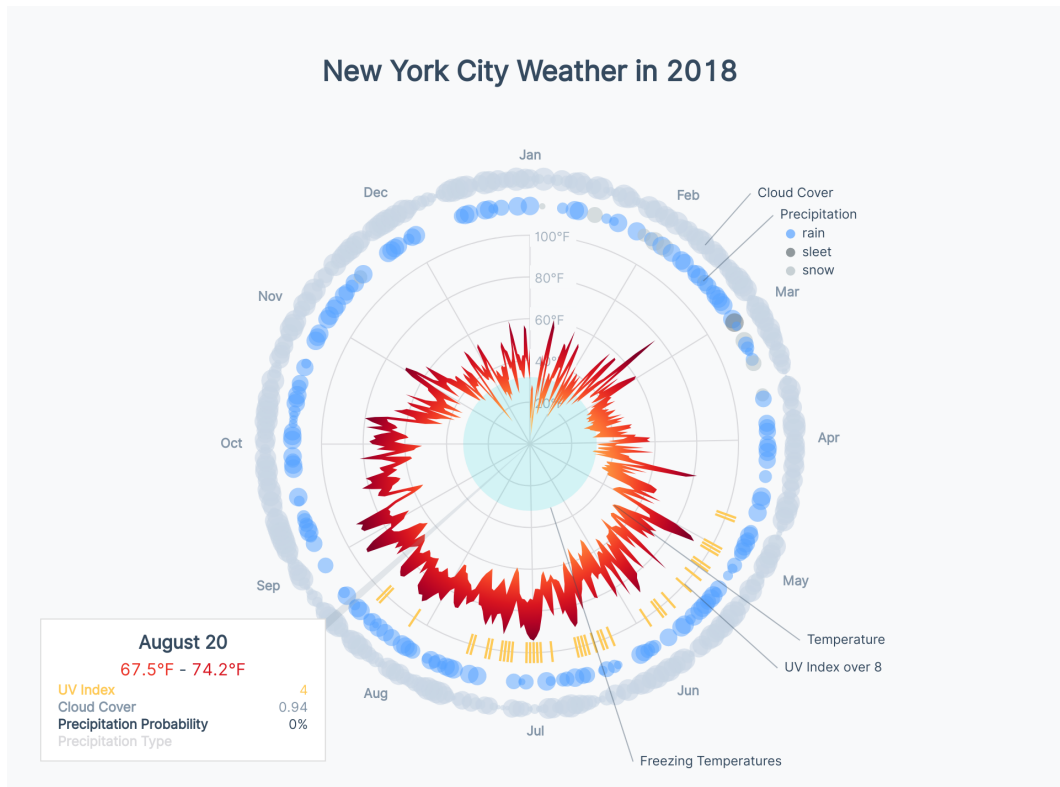


Chart with finished tooltip

Wrapping up

Give yourself a pat on the back, this one was a doozy! Take a step back and look at the visualization we've created. The viewer gets a good sense of the annual weather, and has the ability to explore further, but isn't instantly overwhelmed with information.



Finished chart

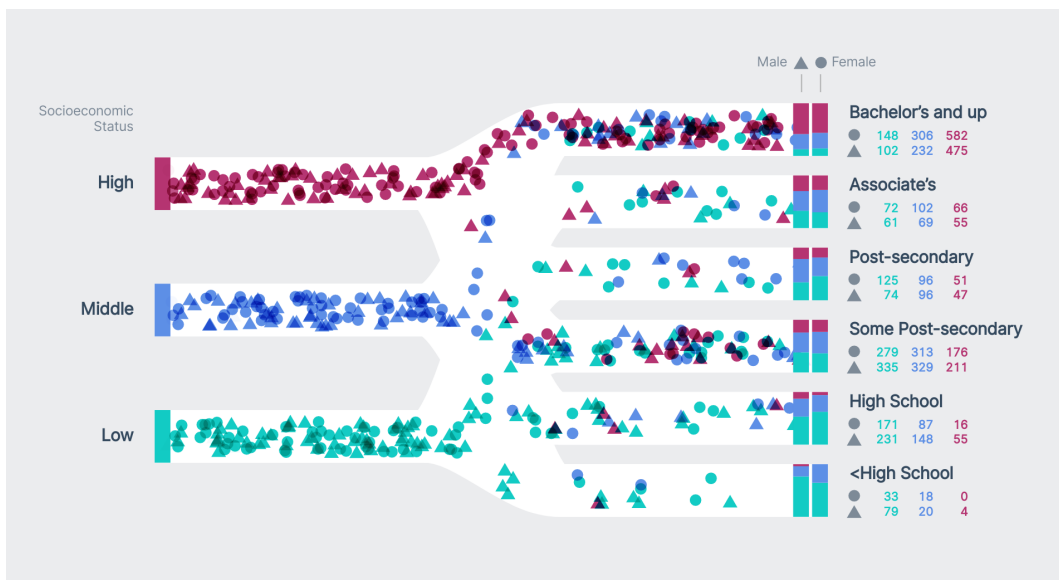
Make sure to show off your chart with friends and on social media! It will be interesting to compare these charts with weather data from different places.

Hopefully this project gave you a better idea of the process involved in making a more complicated chart. Often, we have to research new concepts and learn new parts of the `d3.js` api when making a chart – if we're making a circular chart for the first time, we might need to refresh our trigonometry knowledge and look up ways to draw, for example, arcs.

Animated Sankey Diagram

In this project, we'll be simulating real data and creating an animated diagram to engage our viewers. We won't be using our beloved weather data for this! We'll be using outcomes of a study of 10-year educational achievement for high schoolers in the United States, based on sex and socioeconomic status. For example, 55.3% of the males who grew up with a high socioeconomic status completed at least a Bachelor's degree.

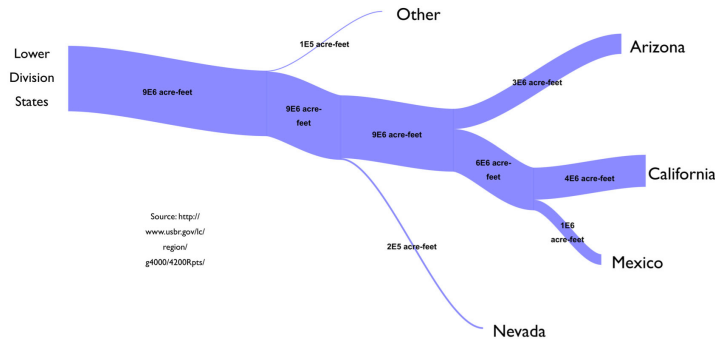
We could make a static data visualization that shows these different probabilities, but our goal here is to engage the viewer. By simulating data and animating it, the viewer gets to “play along” and guess at the outcome before seeing it play out before their eyes.



Finished chart

We'll be mimicking the shape of a [Sankey diagram](https://en.wikipedia.org/wiki/Sankey_diagram)⁷⁶, which shows flows based on the width of paths between points.

⁷⁶https://en.wikipedia.org/wiki/Sankey_diagram



Example Sankey diagram from https://en.wikipedia.org/wiki/Sankey_diagram

Our paths will all be the same width, but using the shape of a Sankey diagram helps us communicate that we’re showing a flow between states.

Getting set up

With your server running (live-server), check out the completed chart at

<http://localhost:8080/12-animated-sankey/completed/>⁷⁷

Our plan is to simulate fake “people” and animate their journey from beginning (socioeconomic status) to end (educational attainment). We’ll represent them with a triangle (for males) or circle (for females), creating new people continuously.

Now that we know what our goal is, let’s open up our draft page at <http://localhost:8080/12-animated-sankey/draft/>⁷⁸ and open the code folder at `/code/12-animated-sankey/draft/`.

Accessing our data

Since this is the first chart we’re making with a new dataset, let’s start by getting acquainted. If we look in the `/code/12-animated-sankey/draft/education.json` file, we’ll see an array of objects.

⁷⁷<http://localhost:8080/12-animated-sankey/completed/>

⁷⁸<http://localhost:8080/12-animated-sankey/draft/>

```

1  [{
2    "sex": "female",
3    "ses": "low",
4    "<High School": 5.4,
5    "High School": 17.1,
6    "Some Post-secondary": 36.2,
7    "Post-secondary": 16.0,
8    "Associate's": 9.3,
9    "Bachelor's and up": 15.9
10 }, {
11    "sex": "male",
12    "ses": "low",
13    "<High School": 10.0,
14    "High School": 26.5,
15    "Some Post-secondary": 35.8,
16    "Post-secondary": 8.7,
17    "Associate's": 6.9,
18    "Bachelor's and up": 12.2
19 }, {
20    "sex": "male",
21    "ses": "middle",
22    "<High School": 2.7,

```

Our data

Each object represents a possible starting point, specified by a sex and a socioeconomic status (ses). The object also has the percent of people in that starting group that attained (at most) various degrees.

If you're curious and want more information about our dataset, follow [the links](#)^a at the bottom of our chart page. They're obtained from a longitudinal study by the the U.S. Department of Education, studying sophomores in high school (2002) until 2012.

^ahttps://nces.ed.gov/programs/digest/d14/tables/dt14_104.91.asp

Let's begin by creating our data accessors, in

/code/12-animated-sankey/draft/chart.js. After we've loaded our dataset, we want to set ourselves up before we start drawing our chart.

Our goal in this section is to be able to create a “person”, using the probabilities in our dataset to decide their educational attainment. We know that each person will need a sex, socioeconomic status, and educational attainment, so let's aim for our people to have the following structure:

```
{  
  sex: 0,  
  ses: 0,  
  education: 0,  
}
```

We *could* use strings for our values (eg. `sex: "female"`), but there are a two main benefits to using indices instead.

- **Performance:** we'll have up to 10,000 objects representing the “people” in our visualization, and strings take up more space than numbers.
- **Scales and math:** two of our metrics are **ordinal** metrics, meaning that they are categorical, but have a natural order. For example, we'll treat **low** socioeconomic status as a *smaller* index than **middle** socioeconomic status. Using numbers instead of strings will come in handy when representing these values in order.

Accessing sex variables

With this data structure in mind, let's create data accessors, starting with **sex**, which we'll use to decide whether a mark is a circle or a square. Our data accessor in this case doesn't help us access our initial dataset, but will help us access the sex of a simulated “person”.

code/12-animated-sankey/completed/chart.js

```
7  const sexAccessor = d => d.sex
```

We'll also want a list of possible sexes, which gives us a list to **sample from** when we generate a person.

code/12-animated-sankey/completed/chart.js

```
8  const sexes = ["female", "male"]
```

Lastly, we'll want a list of ids that correspond to these options. In this case, we'll want the array `[0, 1]` that will correspond to the `["female", "male"]` array, giving females an index of 0 and males an index of 1.

We could hardcode this array, or we could use `d3.range()`⁷⁹ to generate an array of indices, from 0 to the number of options.

code/12-animated-sankey/completed/chart.js

```
9  const sexIds = d3.range(sexes.length)
```

Accessing education variables

We'll create these same variables for **education** values.

code/12-animated-sankey/completed/chart.js

```
11  const educationAccessor = d => d.education
12  const educationNames = [
13    "<High School",
14    "High School",
15    "Some Post-secondary",
16    "Post-secondary",
17    "Associate's",
18    "Bachelor's and up"
19  ]
20  const educationIds = d3.range(educationNames.length)
```

We could create a list of unique education values from our dataset, but hardcoding our `educationNames` array gives us the chance to set the order of values.

⁷⁹<https://github.com/d3/d3-array#range>

Accessing socioeconomic status variables

We'll do these same steps for **socioeconomic status**, keeping the `ses` abbreviation to keep our variable names short.

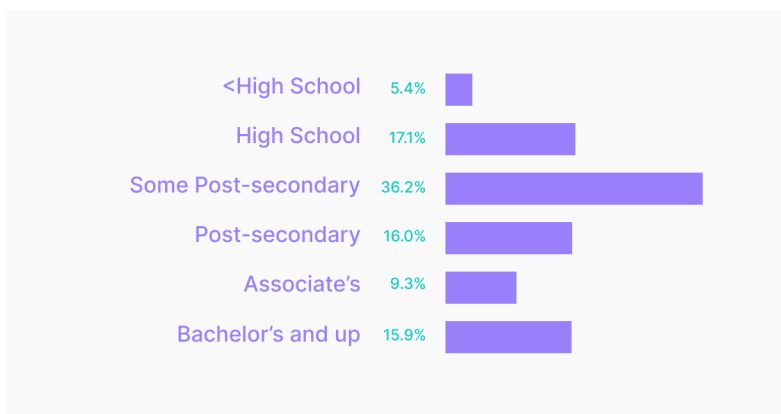
`code/12-animated-sankey/completed/chart.js`

```
22  const sesAccessor = d => d.ses
23  const sesNames = ["low", "middle", "high"]
24  const sesIds = d3.range(sesNames.length)
```

Stacking probabilities

Since we'll be creating several people per second, we'll want our `generatePerson()` function to be as simple as possible. Given their sex and socioeconomic status, we currently know the probability of a person to fall in one of our education “buckets”.

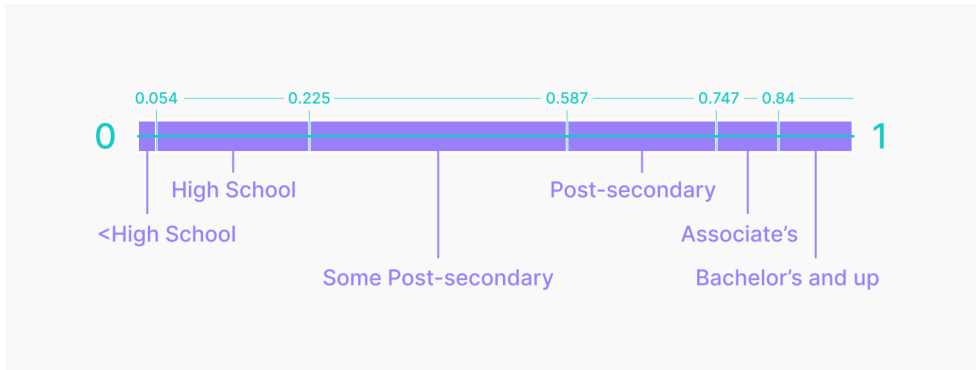
Since these probabilities sum up to 100%, we can stack these probabilities and use a random number to assign a person to a bucket. Let's take the probabilities for females who grew up in a low income household as an example.



Probabilities diagram

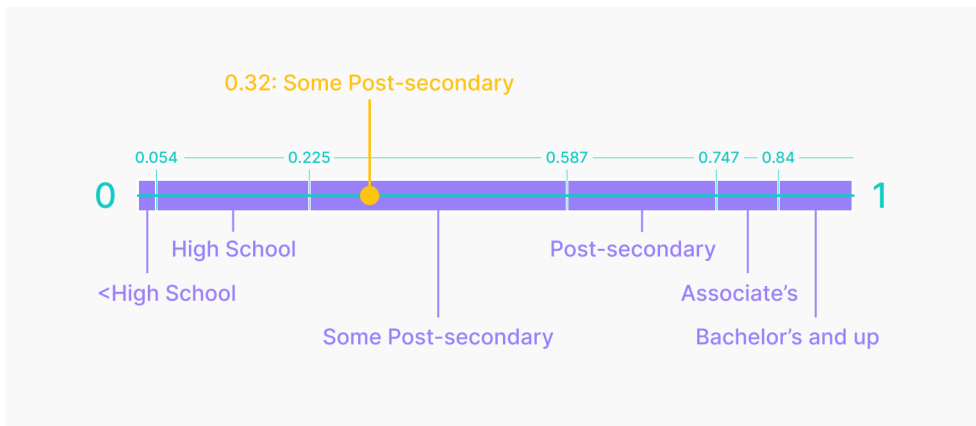
We can take these probabilities and stack them on top of each other, so that each level instead represents the probability that a person achieves *that level or lower*.

The highest level (*Bachelor's and up*), will get the number 1 because there is a 100% chance that a person achieved *that level or lower*.



Probabilities diagram

When we have these stacked probabilities, we'll be able to choose a random number between 0 and 1, which we'll locate on our number-line. For example, if we choose the number 0.32, this person will be placed in the *Some Post-secondary* education bucket.



Probabilities diagram

This is a simple way to choose a bucket, based on pre-existing weights.

Let's set up an object with these stacked probabilities. Our goal is something like this:

```

chart.js:38
▼ {female--low: Array(6), male--low: Array(6), male--middle: Array(6), female--mid
dle: Array(6), male--high: Array(6), ...}
  ► female--high: (6) [0, 0.018000000000000002, 0.21900000000000003, 0.281, 0.34,...
  ▼ female--low: Array(6)
    0: 0.054000000000000006
    1: 0.22500000000000003
    2: 0.58700000000000001
    3: 0.74700000000000001
    4: 0.84000000000000001
    5: 1
    length: 6
    ► __proto__: Array(0)
  ► female--middle: (6) [0.024, 0.113000000000000002, 0.4419999999999995, 0.563, ...
  ► male--high: (6) [0.008, 0.052000000000000005, 0.323, 0.371, 0.446, 1]
  ► male--low: (6) [0.1, 0.365, 0.723, 0.8099999999999999, 0.879, 1]
  ► male--middle: (6) [0.027000000000000003, 0.2, 0.565, 0.6659999999999999, 0.75...
  ► proto : Object

```

Probabilities object structure

To start, we want a consistent way to access the correct set of probabilities, using a “status key” string. Given this object:

```

{
  sex: "female",
  ses: "low",
}

```

we want to generate the string "female--low". Let’s create a function to do that for us.

code/12-animated-sankey/completed/chart.js

```

26  const getStatusKey = ({sex, ses}) => [sex, ses].join("--")

```

Next, we’ll generate an empty `stackedProbabilities` object to populate. We’ll loop over each of our starting points in our dataset, generating the status key and instantiating a `stackedProbability` number.

```
const stackedProbabilities = {}
dataset.forEach(startingPoint => {
  const key = getStatusKey(startingPoint)
  let stackedProbability = 0
  // stackedProbabilities[key] = ...
})
```

Next, we'll loop over each of the education buckets (in order), adding the current probability to the stacked probability, then returning the current sum.

```
const stackedProbabilities = {}
dataset.forEach(startingPoint => {
  const key = getStatusKey(startingPoint)
  let stackedProbability = 0
  stackedProbabilities[key] = educationNames.map((education, i) => {
    stackedProbability += (startingPoint[education] / 100)
    return stackedProbability
  })
})
```

We'll have to add an additional check – if we're looking at the last education bucket, we'll return 1 instead of our running sum. This will help account for rounding errors, where the sum is 0.99 and doesn't completely add up to 1.

code/12-animated-sankey/completed/chart.js

```
28  const stackedProbabilities = {}
29  dataset.forEach(startingPoint => {
30    const key = getStatusKey(startingPoint)
31    let stackedProbability = 0
32    stackedProbabilities[key] = educationNames.map((education, i) => {
33      stackedProbability += (startingPoint[education] / 100)
34      if (i == educationNames.length - 1) {
35        // account for rounding error
36        return 1
37      } else {
38        return stackedProbability
```

```

39         }
40     })
41 }

```

Great! If we `console.log(stackedProbabilities)` after this chunk of code, we'll see that we successfully created our stacked probabilities object.

```

▼ {female--low: Array(6), male--low: Array(6), male--middle: Array(6), female--mid
  dle: Array(6), male--high: Array(6), ...} chart.js:38
  ► female--high: (6) [0, 0.018000000000000002, 0.21900000000000003, 0.281, 0.34,...
  ▼ female--low: Array(6)
    0: 0.054000000000000006
    1: 0.22500000000000003
    2: 0.5870000000000001
    3: 0.7470000000000001
    4: 0.8400000000000001
    5: 1
    length: 6
    __proto__: Array(0)
  ► female--middle: (6) [0.024, 0.11300000000000002, 0.4419999999999995, 0.563, ...
  ► male--high: (6) [0.008, 0.05200000000000005, 0.323, 0.371, 0.446, 1]
  ► male--low: (6) [0.1, 0.365, 0.723, 0.8099999999999999, 0.879, 1]
  ► male--middle: (6) [0.02700000000000003, 0.2, 0.565, 0.6659999999999999, 0.75...
  ► proto : Object

```

Probabilities object structure

Generating a person

Let's put our stacked probabilities to use and create a `generatePerson()` function. We want this function to return an object with a `sex`, `ses`, and `education`.

If we peek at the bottom of our `chart.js` file, we'll see a few utility functions that will help us with some dirty work. For example, there is a `getRandomValue()` function that takes an array of values and returns a random value. Let's use this function to choose a `sex` and `ses` for our person.

```
function generatePerson() {  
  const sex = getRandomValue(sexIds)  
  const ses = getRandomValue(sesIds)  
  
  return {  
    sex,  
    ses,  
    education: "?",  
  }  
}
```

If you're confused about the `{sex}` syntax, we're using ES6 **Object shorthand** notation. `{sex}` is the same thing as `{sex: sex}`. If you want to read more, [here's a great source](https://tylermcginis.com/shorthand-properties/)^a.

^a<https://tylermcginis.com/shorthand-properties/>

But how do we choose our person's education? First, we'll generate a `statusKey` and grab the matching stacked probabilities.

```
function generatePerson() {  
  const sex = getRandomValue(sexIds)  
  const ses = getRandomValue(sesIds)  
  const statusKey = getStatusKey({  
    sex: sexes[sex],  
    ses: sesNames[ses],  
  })  
  const probabilities = stackedProbabilities[statusKey]  
  
  return {  
    sex,  
    ses,  
    education: "?",  
  }  
}
```

Our probabilities array will look something like:

```
[0.054, 0.225, 0.587, 0.747, 0.84, 1]
```

We can generate a random number using `Math.random()` and use `d3.bisect()` to find the index where that number will “fit” in our probabilities array.

```
function generatePerson() {  
  const sex = getRandomValue(sexIds)  
  const ses = getRandomValue(sesIds)  
  const statusKey = getStatusKey({  
    sex: sexes[sex],  
    ses: sesNames[ses],  
  })  
  const probabilities = stackedProbabilities[statusKey]  
  const education = d3.bisect(probabilities, Math.random())  
  
  return {  
    sex,  
    ses,  
    education,  
  }  
}
```

Let’s test this out by generating a few test people:

```
console.log(generatePerson())  
console.log(generatePerson())  
console.log(generatePerson())
```

We’ll see something like this in the console:

```
▶ {sex: 1, ses: 2, education: 4} chart.js:56  
▶ {sex: 0, ses: 1, education: 5} chart.js:57  
▶ {sex: 1, ses: 0, education: 2} chart.js:58
```

Test people

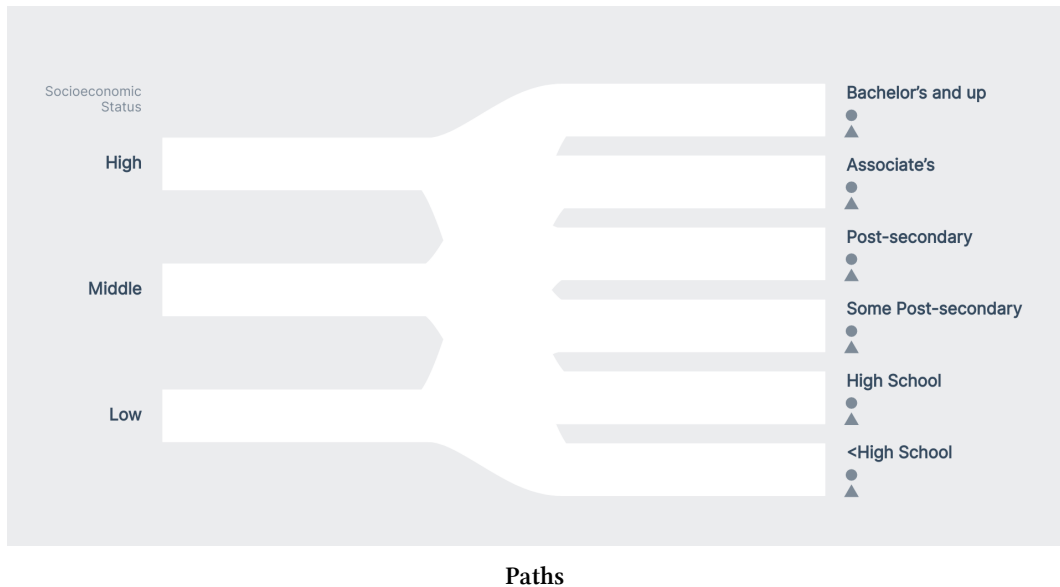
Great! Now we can generate people who follow the rules of our dataset.

Drawing the paths

Before we draw our people, we'll need paths for them to follow. Let's prepare by adding a `pathHeight` constant to our existing `dimensions` object. 50 pixels tall is a good height that will fit a lot of people at once, but not push lower categories off the page.

```
let dimensions = {  
  width: width,  
  height: 500,  
  margin: {  
    top: 10,  
    right: 200,  
    bottom: 10,  
    left: 120,  
  },  
  pathHeight: 50,  
}
```

We have three starting y-positions and six ending y-positions – our goal is to connect each of these with a curved line that our markers will follow.



X scale

We'll need to create some scales first in our **Create scales** step.

Let's start by creating an x scale that converts a person's progress (from left to right) into an x-position. We'll represent this progress with a number from 0 (not started yet) to 1 (has reached the right side).

code/12-animated-sankey/completed/chart.js

```
108  const xScale = d3.scaleLinear()
109    .domain([0, 1])
110    .range([0, dimensions.boundedWidth])
```

We don't want our markers moving beyond the left or right side of our paths, so we'll `.clamp()` this scale. This way, the smallest number our scale will return is 0 and the largest number is 1, even if we give it a progress of 10.

code/12-animated-sankey/completed/chart.js

```

108   const xScale = d3.scaleLinear()
109     .domain([0, 1])
110     .range([0, dimensions.boundedWidth])
111     .clamp(true)

```

Y scales

Next, we'll create a scale that will convert from a socioeconomic id to a y-position. We want these paths to be evenly spaced between our bounds, but to still fit inside.

We can achieve this by padding our scale's domain, making it span $[-1, 3]$ instead of $[0, 2]$. This way, our *real* socioeconomic status ids will fit inside our bounds.



y scale diagram

We'll also want our scale's domain to be *backwards*: $[3, -1]$ instead of $[-1, 3]$ because we want **the highest y position (closer to the bottom) to correspond to the lowest id**.

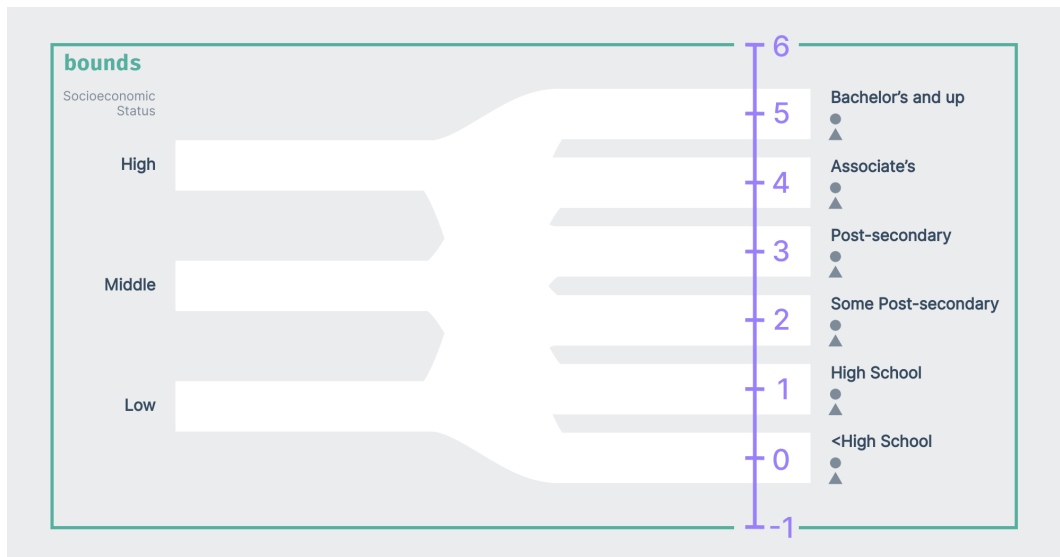
code/12-animated-sankey/completed/chart.js

```

113   const startYScale = d3.scaleLinear()
114     .domain([sesIds.length, -1])
115     .range([0, dimensions.boundedHeight])

```

Let's create a similar scale for our educationIds on the right side of our chart.



y scale diagram, right

code/12-animated-sankey/completed/chart.js

```

117   const endYScale = d3.scaleLinear()
118     .domain([educationIds.length, -1])
119     .range([0, dimensions.boundedHeight])

```

Drawing the paths

Let's use our scales to draw some paths. Our goal is to draw a path like this one:



Example path

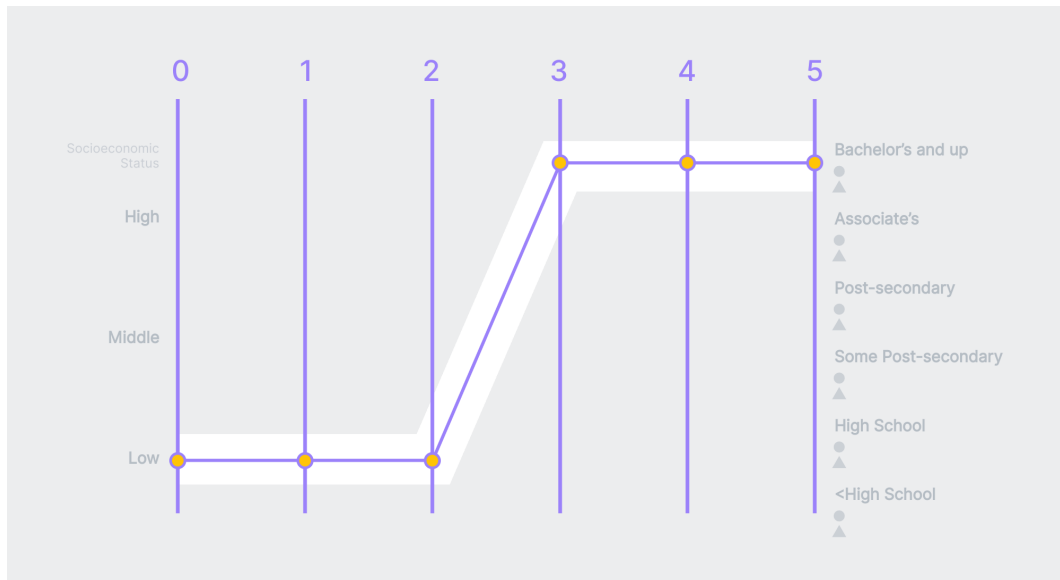
between each of our starting points and each of our ending points.

Since these shapes aren't linear, we'll draw them using `<path>`s by using `d3.line()` to create a `d` string attribute generator. Our line generator will take an array of six identical arrays. For example, we might pass it this input:

```
[
  [0, 5],
  [0, 5],
  [0, 5],
  [0, 5],
  [0, 5],
  [0, 5],
]
```

The first item in each of these arrays (0) is the **socioeconomic status id** (starting point) and the second item (5) is the **education id** (ending point).

Our link generator will return the *starting y position* for the first 3 arrays, and the *ending y position* for the last 3 arrays.



Path parts diagram

The reason we want to repeat this array 6 times is to devote one-fifth of our horizontal space to the y-position transition. If we only had four identical arrays, we would be devoting *half* of our horizontal space to the transition, which would make our final chart way more chaotic. Our markers would spend one third of their time moving up and down!

Let's create that line generator:

code/12-animated-sankey/completed/chart.js

```

133  const linkLineGenerator = d3.line()
134    .x((d, i) => i * (dimensions.boundedWidth / 5))
135    .y((d, i) => i <= 2
136      ? startYScale(d[0])
137      : endYScale(d[1])
138    )

```

We'll need to create that six-item array for each permutation of starting and ending ids. We'll map over each of our starting ids and *also* each of our ending ids, creating that six-item array for each loop. We'll pass the result to `d3.merge()`, which will flatten these into one array.

code/12-animated-sankey/completed/chart.js

```

140     const linkOptions = d3.merge(
141       sesIds.map(startId => (
142         educationIds.map(endId => (
143           new Array(6).fill([startId, endId])
144         ))
145       ))
146     )

```

Our resulting lineOptions array will look like this:

[chart.js:134](#)

```

(18) [Array(6), Array(6), Array(6), Array(6), Array(6), Array(6), Array(6), Array(6), Ar
▼ ray(6), Array(6), Array(6), Array(6), Array(6), Array(6), Array(6), Array(6), Ar
  ray(6), Array(6), Array(6)] 1
  ▼ 0: Array(6)
    ▶ 0: (2) [0, 0]
    ▶ 1: (2) [0, 0]
    ▶ 2: (2) [0, 0]
    ▶ 3: (2) [0, 0]
    ▶ 4: (2) [0, 0]
    ▶ 5: (2) [0, 0]
    length: 6
    __proto__: Array(0)
  ▼ 1: Array(6)
    ▶ 0: (2) [0, 1]
    ▶ 1: (2) [0, 1]
    ▶ 2: (2) [0, 1]
    ▶ 3: (2) [0, 1]
    ▶ 4: (2) [0, 1]
    ▶ 5: (2) [0, 1]
    length: 6
    __proto__: Array(0)
  ▶ 2: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 3: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 4: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 5: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 6: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 7: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 8: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 9: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 10: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 11: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 12: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 13: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 14: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 15: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 16: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  ▶ 17: (6) [Array(2), Array(2), Array(2), Array(2), Array(2), Array(2)]
  length: 18
  __proto__: Array(0)

```

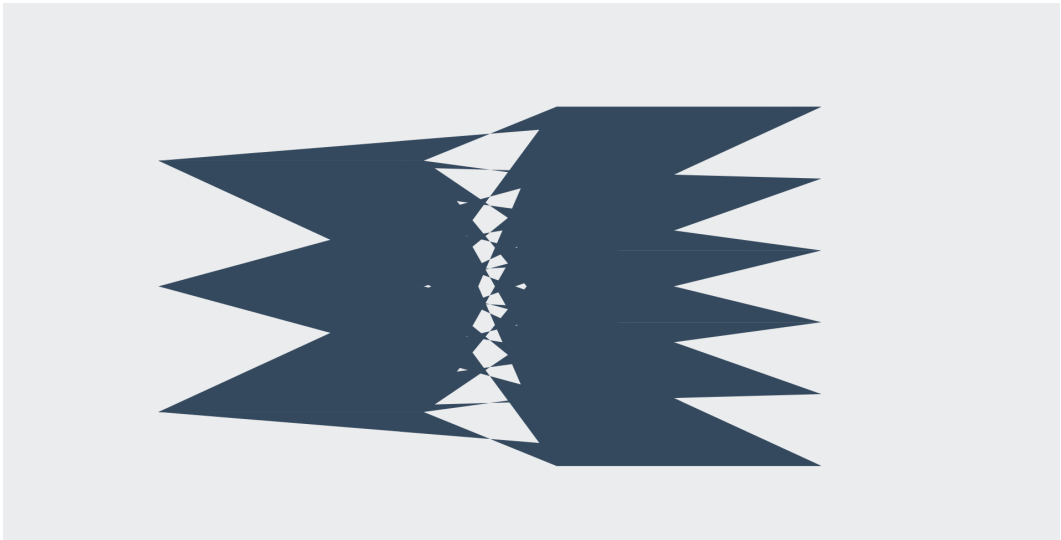
lineOptions structure

Great! Now we just need to create a `<path>` for each of our `linkOptions` and generate their `d` attribute strings using our line generator.

code/12-animated-sankey/completed/chart.js

```
147   const linksGroup = bounds.append("g")
148   const links = linksGroup.selectAll(".category-path")
149     .data(linkOptions)
150     .enter().append("path")
151     .attr("class", "category-path")
152     .attr("d", linkLineGenerator)
153     .attr("stroke-width", dimensions.pathHeight)
```

Hmm, something is wrong here.



Our paths

We'll need to set the `fill` and `stroke` colors of our paths in the `styles.css` file.

```
.category-path {  
  fill: none;  
  stroke: white;  
}
```

Much better! Hmm, some of these paths have sharp edges.



Our paths, styled

Let's add an interpolator function to our line generator to smooth our paths.

code/12-animated-sankey/completed/chart.js

```
133  const linkLineGenerator = d3.line()  
134    .x((d, i) => i * (dimensions.boundedWidth / 5))  
135    .y((d, i) => i <= 2  
136      ? startYScale(d[0])  
137      : endYScale(d[1])  
138    )  
139    .curve(d3.curveMonotoneX)
```

There we go!



Our paths, styled and curved

Now our paths are cleaner and flow better.

Labeling the paths

In our **Draw peripherals** step, let's add labels so that our viewers know what the start and end of each path is.

Start labels

First, we'll label each possible start, positioning our labels within a group that is 20 pixels left of the beginning of our **bounds**. Let's first create a `<g>` element that positions our labels' x-positions.

code/12-animated-sankey/completed/chart.js

```
157   const startingLabelsGroup = bounds.append("g")  
158     .style("transform", "translateX(-20px)")
```

Then we'll create each individual label, using the utility function `sentenceCase()`, defined at the bottom of our file, to format our socioeconomic labels.